



The Lazy Developer

10

MODULE

Scaling, Monitoring, & Performance Metrics

A prototype that works for you and a friend on a Saturday night can crumble the moment 500 real people hit “Sign up.” Scaling and observability are the guard-rails: they stop “success disasters,” and help you notice problems before users do.

Table of Contents

01. From One User to Ten Thousand (10 min)
02. Traffic Bumpers: Simple Rate Limiting & One Job Queue (10 min)
03. See It Happening: Logs, Metrics & One Dashboard (13 min)
04. Ship Carefully: Feature Flags & Instant Rollbacks (10 min)
05. Keep the Lights (and Wallet) On: Auto-Scaling & Daily Health Checks (10 min)

Forecast the Traffic & Stage the Launch

Think of your prototype as a single-lane bridge. On a quiet day it's fine, but one tour-bus too many will snap the beams. Section 1 is about measuring the expected traffic before the buses arrive so you can widen the bridge just enough, not build a four-lane highway you'll never use, and definitely not leave it as a rickety plank. That foresight pays off three ways:

1. Money: you size servers to real demand instead of "just in case" over-spend.
2. Stability: you fix the most likely failure point early, avoiding 2 a.m. fire-drills.
3. Credibility: a smooth launch tells customers (and investors) you know what you're doing.

The rest of this module with topics like rate-limiting, queues, and metrics plugs into the numbers you establish here. So, let's get those numbers right.

Turn "Hope" Into Numbers

Before you open a load-testing tool, map a typical user journey and translate it into maths you can defend.

Why this matters

"Lots of people might sign up" isn't a capacity plan. A single, defensible requests-per-second (RPS) figure becomes the north-star for everything from database sizing to cloud costs. Even if reality is 2x your guess, you'll still be 10x more prepared than winging it.

Steps

1. List the user actions. Capture one end-to-end visit: homepage '•' browse '•' checkout.
2. Attach call counts. How many API or DB calls per action? Static images from a CDN don't count toward backend load.
3. Choose a realistic crowd.
4. Run the back-of-napkin formula.

peak RPS = (peak concurrent users x requests per user) / session length in seconds

Example: 500 users x 30 requests ÷ 600 s = 25 RPS.

5. Write that number down. Everything else keys off it.

Risk if skipped: you'll either launch under-provisioned (downtime) or over-provisioned (credit-card shock).

AI prompt to try:

"I'm trying to get my head around how much load we can handle. Is this a feasible scenario for our app? Given 1 000 daily users at 30 requests each, estimate peak RPS and recommend the lowest-

cost cloud solution setup that handles it with p95 latency < 300 ms.”

Spot the First Bottleneck (No Hunches, Only Data)

Your shiny new RPS tells you how much traffic, but not where it will break first. Guessing leads to wasted effort like profiling the CPU when the DB is the real choke. Instead, test systematically.

Why this matters

Every app has a weakest link: CPU spikes, memory leaks, slow queries, or bandwidth gaps. Finding it early means a one-evening fix instead of an emergency rewrite when users complain.

How to uncover it

Resource to test	What breakage looks like	Five-minute confirmation
CPU	90 %+ during load !’ server queue	top, CloudWatch CPU graph
Memory	builds “OOM-Killed” container restarts	watch RAM usage while hitting /
Database locks	All requests freeze during bulk	api/heavy run 50 parallel writes in staging
Bandwidth	export Images crawl on café Wi-Fi	download a 5 MB file from staging

Pick one suspect, write a tiny load script (Locust, k6, or ask AI) and collect hard numbers. Fix, rerun, then move to the next.

Benefit: incremental tuning; no premature optimisation rabbit-holes.

AI prompt to try:

“Come up with a testing plan that can help me find a way to see if there are any bottlenecks in the system. Start with the 4 main areas of CPU, Memory, DB, and bandwidth. Perhaps you can generate a Locust script simulating 500 users running login '• browse '• checkout so I can watch CPU and DB metrics.”

1.3 Roll Out in Controlled Layers

Even perfect forecasts can’t model human excitement. A TikTok mention can 20x traffic overnight. Layered rollouts let you absorb surprises gracefully.

Why this matters

A full-blast public release is high-stakes roulette: if something goes wrong, everyone sees it. A phased launch lets you catch issues with a friendly audience first, then dial-up traffic once dashboards stay green.

Three practical layers

1. Private link – share a secret URL with teammates; squash obvious bugs.
2. Invite-only beta – onboard 50–100 eager users; watch error logs, latency graphs, and cost dashboards. AI can help you build all this if you know what metrics make the most sense to monitor.
3. Metered public launch – open sign-up to the world but throttle new accounts (e.g., 200/hour) until metrics stay flat for 24 h.

Benefit: user feedback arrives in manageable waves; rollback scopes stay small.

AI prompt to try:

“Draft a two-week launch plan: private link day 1, invite-only list by day 4, public launch day 10, with monitoring checkpoints at each step.”

1.4 What “Done” Looks Like

- Documented peak-RPS calculation (spreadsheet or README table).
- One confirmed bottleneck fix—backed by load-test numbers.
- Three-step rollout plan copied into your project board, including who watches which dashboard.

2. Traffic Bumpers – Simple Rate-Limiting & a Single Job Queue

Your app is now sized for the expected load, but the internet rarely behaves as expected. A viral share, a mis-configured script, or a malicious bot can turn neat forecasts into stampedes. Section 2 adds two low-friction “bumpers” that absorb those shocks:

1. Rate-limiting – caps how fast any one source can hit you.
2. A background job queue – lifts slow, heavy work off the critical request-response path.

Put them in place and the most common traffic spikes turn into shrug-worthy blips instead of outage post-mortems.

2.1 Put a Speed-Limit Sign on the API

Why this matters

Without a speed limit, one buggy browser tab set on “auto-refresh” can chew through CPU, memory, and database connections locking out honest users in seconds. A single rule at the edge saves you from debugging the wreckage.

Practical recipe

1. Pick an entry point you already run.
2. Set one conservative rule.
3. Return helpful headers.
4. Log the events.

Risk if skipped: your first marketing push might translate to “works great for five minutes, then 503 errors for everyone.”

AI prompt to try

“Generate a config that limits email sending to 100 requests per minute per IP, allows a burst of 20, and sends Retry-After headers.”

2.2 Move Heavy Lifting to a Queue

Why this matters

File uploads, PDF exports, email sends or anything that waits on disk I/O or third-party APIs blocks the web request until it’s done. Users feel that as an eternal spinner; servers feel it as memory pressure. A queue turns those tasks into fire-and-forget: the user gets an instant 202-Accepted, and a background worker finishes the job without holding up the line.

Practical recipe. Did you know I was going to say "Talk to AI about this"?

2.3 Catch & Throttle Obvious Abuse Early

Why this matters

Credential-stuffing bots and sign-up spammers don't just hurt your app-they can rack up real costs (email, SMS) and poison analytics. A few heuristics block 80 % of junk with almost no engineering overhead.

Practical recipe

Pattern to watch	Quick rule of thumb	First-line defence
Too many sign-ups	> 5 new accounts per IP per hour	30-minute IP temp-ban + CAPTCHA
Login brute-force	10 failed logins in 5 min	Incremental backoff: 1->2->4 s delay
Credential stuffing	Same IP hits /login > 100x/hr with random emails	Block IP for 24 h; flag account IDs

Keep the rules blunt and transparent. You can refine them later with rate graphs and anomaly detection once Section 3's metrics are live.

AI prompt to try

"Add middleware (remember that bouncer of the club?) that blocks an IP for 30 minutes after 5 failed sign-ups in 10 minutes and returns JSON {error:'Too many registrations'}."

2.4 What "Done" Looks Like

- Ø=P! One active rate-limit rule at the API edge, logged and visible.
- Ø=Üî Redis-backed queue + one worker running in staging, with retry and dead-letter handling.
- Ø=PÑ Basic abuse heuristics (sign-up storm & brute-force login) live and tested.
- Ø=ÜÈ Two sanity metrics: "429 responses per minute" and "tasks waiting in queue" piped to a dashboard (you'll wire that up fully in Section 3).

Lock these bumpers in place now, and sudden traffic spikes will feel like speed bumps, not head-on collisions freeing you to focus on visibility and alerting in the next chapter.

Picture your app as a little shop on a quiet street. In Sections 1 and 2 you estimated foot-traffic and installed a door-counter plus speed-bumps. Section 3 is the CCTV and the “manager rings your phone at night” system. The goal is simple:

Notice problems while they’re starting, not when your first angry tweet appears.

Below we unpack four building-blocks being logs, metrics, dashboards and alerts and with plain-English explanations and shoestring options. Each block ends with a copy-ready AI prompt so you can have the AI generate the code or config for you.

Collect What Happened with Logs

What’s a log?

A log is just the app writing down what it did: “10 Jun 14:03, GET /checkout, OK, took 82 ms.” In an outage, logs are your black-box recorder.

Why “structured” logs?

If every line follows the same machine-readable shape (usually JSON), you can filter or graph them later without messy text searches.

```
{
  "ts": "2025-06-26T14:00:11Z",
  "level": "INFO",
  "method": "GET",
  "route": "/api/cart",
  "status": 200,
  "latency_ms": 82,
  "user": "u_3f9a17"    // hash, not email
}
```

Cheapest place to keep them

- Local file with log-rotation (zero cost, fine for the first few thousand users).
- Grafana Cloud free tier — 50 GB of logs a month, hosted for you ([grafana.com][1], [grafana.com][2]).
- Any cloud provider’s “free-ish” log bucket (e.g., AWS CloudWatch Free Usage).

AI prompt (see more detail about this prompt at the end of this section)

“Generate a logger that outputs one JSON line with ts, level, method, route, status, latency_ms, user (SHA-256 hash). Rotate the file daily.”

Watch the Vital Signs with Metrics

Logs vs. metrics

Logs = individual stories.
Metrics = the big picture — numbers that change over time and plot as a line chart.

The Four Golden Signals (start here)

Metric	Plain-English meaning	Why you care
Requests per second	How busy are we?	Detect traffic floods or total silence
Median latency	Typical wait time	Shows day-to-day “feel” for users
p95 latency	Slowest 5 % of requests	Finds tail-end hiccups your boss always notices
Error rate (4xx + 5xx)	% of requests that fail	Immediate clue to bugs or DB trouble

Key term — p95 (95th percentile): if p95 = 800 ms, then 95 % of requests are faster than that; 5 % are slower.

A free way to collect them

1. Add a tiny Prometheus client library (one import, a few lines).
2. Expose /metrics—Prometheus scrapes it every 15 s.
3. Point Grafana Cloud free tier at the scrape URL (10 000 active series included).

AI prompt (see more detail about this prompt at the end of this section)
“I was told to ask you to create a Python FastAPI snippet that records request_latency_seconds as a Prometheus histogram with buckets 0.05, 0.1, 0.25, 0.5, 1, 2, 5. After you do that, explain to me in plain English what that is for!”

3.3 See It at a Glance with One Dashboard

Why a dashboard?

Graphs turn “something feels slow” into “checkout latency jumped to 1.2 s at 18:07.” It’s the cockpit display everyone can understand.

Two cost-zero paths

Option	What you do	Rough cost
Grafana Cloud (hosted)	Sign up !’ copy your Prometheus URL !’ import starter dashboard	Free tier (10 k metrics, 3 users) ([linkedin.com][3])
Home-rolled HTML page	Ask Cursor to make a static HTML + Chart.js file that reads a small CSV you upload daily	\$0 hosting on GitHub Pages or Netlify

Either way, track the four metrics and colour the chart background red when error-rate > 2 %.

AI prompt (see more detail about this prompt at the end of this section)
“I have signed up for Grafana’s free tier. Can you produce a Grafana dashboard JSON (and teach me how to import it) that graphs RPS, median latency, p95 latency, error rate, and colours the error

panel red when rate > 2 % for 5 min.”

3.4 Get a Nudge with Alerts & Public Status

Why alerts matter

Graphs are useless while you’re asleep. An alert rule decides when to bother you (“only if it’s bad for 5 minutes, not a blip”).

No-budget options

- Grafana Cloud Alerting — e-mail or Slack on any threshold (included in free tier).
- Tiny cron script — every minute curl /health; if it fails 3x, mail -s "API down" you@example.com.
- Instatus free status page — up-down monitoring + public “green / yellow / red” badge for your users. Supe simple and FREE to setup. Check out mine at haf.instatus.com

AI prompt (see more detail about this prompt at the end of this section)
“Write a Bash script run by cron that hits https://myapp.com/health every minute and sends an email via sendmail after three consecutive failures.”

3.5 Checkpoint — What “Good Enough” Looks Like

- ' Structured JSON logs land in a single file or Grafana Cloud.
- ' /metrics endpoint exposes the four signals.
- ' One dashboard (Grafana Cloud or static HTML) shows those signals in real time.
- ' One alert pings you when error-rate > 2 % for 5 minutes.
- ' (Optional) Public status page on Instatus so users see issues before hitting support.

With eyesight in place, you can add new features confidently because Section 4 will teach you to toggle them on and off without redeploy.

More detail about the prompts

What Each Sample Cursor Prompt Actually Produces —and Why It’s Useful

#	Prompt (copy-ready text)	Behind-the-scenes Cursor returns outcome	Why it’s handy in real life
1	“Generate a Node.js winston logger that outputs one JSON line with ts, level, method, route, status, latency_ms, user (SHA-256 hash). Rotate the file daily.”	Cursor returns a short JavaScript module that: • Imports winston and its format.json() helper. • Defines a custom formatter that	

Ship Carefully with Feature Flags & Instant Rollbacks

Imagine you've just added a "Download my data" button. It worked fine on your laptop, yet when 5 000 people click it at once the database groans and page loads everywhere slow to a crawl.

A feature flag is a safety-valve: you can turn that button on only for staff, then for 5 % of real users, and if trouble brews flip it straight back off without another deploy. In other words, flags let you experiment in production without gambling with all your customers at once.

In this section we'll cover:

1. What a flag is and where it should live
2. Planning a low-risk rollout that starts small and grows safely
3. Smoothing the user journey so the test feels invisible to everyone involved
4. Rollbacks—manual and automatic—and the human factors you should think through
5. Natural-language Cursor prompts you can paste verbatim to build each piece

What, Why & Where of Feature Flags

A feature flag is a simple ON / OFF (sometimes a percentage slider) that lives outside your code. The app reads the flag at runtime and decides which path to follow.

Why bother?

- Launch small, learn fast — expose your idea to 50 users, collect feedback, iterate.
- Shield revenue — if the new path doubles query time, you can kill it in ten seconds.
- Separate deploy from release — ship code any day, but reveal it when marketing is ready.

Where should the switch live?

For most indie or small-team projects the easiest place is an "Admin %p Feature Toggles" page inside the app itself. You (or any teammate with admin rights) log in, tick a box, hit save and you're done. That beats raw database edits and avoids paying for a SaaS tool on day one.

AI prompt — plan the admin page

"Design a simple admin page called 'Feature Toggles' with a list of flags (name, on/off switch, rollout slider), a 'Save' button, and a note saying changes take effect within one minute."

Above is the feature flags admin page at Quiki.

Planning a Gentle Roll-Out

Let's use our running example: "Download my data".

1. Clarify the goal.
“I want users to be able to download everything we store about them, but I’m unsure how much load the export creates.”
2. Start with insiders.
Step you tell AI: “Enable the feature only for staff accounts so we can click around and watch the server graphs.”
3. Expand to a thin slice of real users.
“Turn the button on for 5 % of customers chosen at random.”
Why random? It gives you a statistically fair mix—heavy users, casual users, different time zones—so you see problems early.
4. Gather numbers & human feedback.
Lean on the dashboard you built in Section 3: latency, error rate, CPU, plus a quick support-inbox scan for confusion.
5. Scale to 100 % or roll back.
If graphs stay green and users like it, slide the rollout to 100 %. If not, set it back to 0 % and regroup.

AI prompt to describe the ask in plain English

“Help me add the ‘Download my data’ button so that only 5 % of users see it for the first 24 hours, and make sure I can turn it off instantly from the admin panel if the server slows down.”

Walking Through the Roll-Out Journey

You flip the flag to Staff Only at 09:00. Jane from Support clicks the new button; it takes 12 seconds, but she’s patient. You notice CPU climbs to 75 % for a minute—acceptable.

At lunch you widen the flag to 5 % of users. One customer tweets happily they’ve grabbed their archive; ten minutes later your error-rate panel nudges from 0.3 % to 0.9 %. Still green.

Support remains quiet. You let it run overnight. Next morning metrics are flat—great! You move the slider to 100 % and announce the feature in your newsletter. Mission accomplished.

If, instead, the overnight error rate had spiked, you’d have pulled the slider back to 0 %, posted a quick status update (“Temporarily disabling data download while we improve performance”), and no widespread outage would follow.

Rollbacks: Why, How, and the Human Side

Manual rollback (you untick the flag) is perfect when you’re awake and watching the graphs.

Automatic rollback is your seat-belt at 3 a.m.

Why automate?

Servers don’t wait for office hours. If the error-rate crosses 3 % for five straight minutes at midnight, a script can flip the flag off faster than PagerDuty can wake you.

Ramifications to consider

- User perception
Someone may wonder, “Where did the button go?” Publish a short status-page entry or in-app banner saying the feature is temporarily paused for upgrades.
- Data integrity
If the new path wrote to new tables, be sure rolling back doesn’t orphan half-finished records. Before launch, define what “undo” looks like.
- Alert fatigue
Tune your auto-rollback so it needs sustained trouble (e.g., 5 minutes of errors) not a 30-second blip, otherwise the flag may flap ON-OFF all night.

AI prompt

“Create a background task that checks every minute: if error-rate stays above 3 % for the last five minutes, switch the ‘Download my data’ flag off, post a message in our #ops Slack channel, and send me an email summary.”

AI Prompts Rewritten for Humans, Not Hackers

Scenario you describe to AI	What the AI will deliver
“Add a new feature toggle called ‘download_data’ to the admin page with an on/off switch and a rollout slider. I want to test the download feature with only employees first. Employees have emails ending @ourcompany.com. How can I set that up? After two hours, widen the rollout to 10 % of everyone automatically, can you schedule that?”	A form added to your existing admin template, plus the tiny model code needed to store the flag.
“If more than 50 errors happen in five minutes after enabling the feature, switch it off and send me a Slack DM. Write a friendly banner that appears when we disable the feature, telling users we’re improving performance and it’ll be back soon.”	A snippet that checks the logged-in user’s email domain and shows the button only to matching accounts.
	A small, scheduled job (cron or cloud function) that updates the flag at the two-hour mark.
	A monitoring script that reads the error metric, counts failures, flips the flag via your admin API, and hits the Slack webhook you give it.
	Ready-to-paste HTML/React component plus a one-line condition: <code>if !feature.enabled('download_data') showBanner()</code> .

Quick Success Checklist

- ' Admin “Feature Toggles” page exists; no direct DB poking required.
- ' First feature (“Download my data”) lives behind a flag, staff-only for day one.
- ' Gradual rollout slider tested—5 % ! 100 %.
- ' Auto-rollback task flips the flag off if errors climb; you receive the alert.
- ' User-facing banner/status page explains any temporary withdrawal.

Keep the Lights (and Wallet) On with Auto-Scaling & Tiny Health-Checks

Think of your app as a food-truck at a festival:

- At lunchtime the queue is 30 people long and you wish you had a second chef.
- At 3 a.m. the street is empty and then you wish you hadn't paid both chefs to stand around.

Auto-scaling is an extra chef who shows up automatically when the queue grows and clocks off when things calm down. Daily health-checks are your quick walk-round each morning: "Are the fridges cold? Any new charges on the credit card?"

Everything here is doable with click-through cloud consoles or copy--paste scripts with no deep DevOps needed. These systems have come a long way from a few years ago where you needed to get certified just to use them.

Automatic Right-Sizing: More Muscle Only When You Need It

What's happening under the hood?

Imagine each copy of your app (a container) has a tiny CPU gauge. You tell the cloud:

- "If that gauge stays over 70 % for 5 minutes, start one more copy."
- "If it drifts below 30 % for 10 minutes, shut one copy down but please always leave at least one running."

The cloud watches the gauge every few seconds and follows your rule. That's it.

AI prompt:

"Please write the exact steps (in plain English and any needed JSON) that set my AWS app to add one extra copy when average CPU is above 70 percent for five minutes, and remove one copy when below 30 percent for ten minutes — always keeping at least one copy alive. Assume I'm using ECS Fargate."

When you paste that into the chat the AI will:

1. List the clicks in the AWS console and
2. Give you a ready-to-paste JSON policy for the "Auto Scaling ! Target Tracking" screen.

Run a five-minute load-test (Section 3 has one) and watch a second task appear, then disappear later that evening giving you proof it works.

Stop Bill-Shock Before It Starts

Cloud costs tick up the moment auto-scaling fires. A budget alert is a simple alarm clock:

1. You choose a “too-much” number, say £150 for this month.
2. The cloud e-mails you (and maybe your co-founder) at 80 % of that number.
3. You can then tune the auto-scaling rule or investigate any surprise traffic.

AI prompt:
“Show me, step-by-step, how to set a monthly AWS budget cap of £150 and get an e-mail alert when the forecasted spend reaches 80 % of that.”

AI's reply will include the exact menu path (Billing ! Budgets ! Create Budget) plus a copy--paste example of the alert e-mail text.

The Ten-Minute Daily Health Ritual

You don't need to be “on call” 24x7, but a short checklist keeps small cracks from becoming weekend fires.

1. Open yesterday's graph screenshot (Grafana ! choose “Yesterday”).
Are the coloured lines roughly flat?
2. Look at the “Alerts” e-mail folder.
Any overnight budget or health warnings?
3. Check backups — most databases show a green tick; it takes two clicks.
4. Skim the auto-scaling history (AWS ! “Scaling Activities”).
Did ten extra copies appear at 2 a.m.? If yes, why?
5. Write one sentence in your Ops Log
“2025-07-04 — Latency flat, backups OK, 1 auto-scale event, spend forecast £62/£150.”

Time spent: about the length of a coffee-break.

AI prompt
“Create a simple daily-ops checklist I can paste into Notion: check graphs, alerts, backups, scaling events, and a free-text notes line.”

Expanded AI Prompts & What They Deliver

Friendly prompt you paste into AI	What AI gives you (in plain words)	Why it's useful today
“Explain, like I'm five, how to turn on auto-scaling for my one-container app on AWS Fargate. Write a Slack message template that the cloud can post when it has to spin up a new container, telling with that 70 / 30 CPU rule.”	A bullet-list of AWS console clicks plus the exact JSON for the scaling policy — already filled with your 70 % / 30 % numbers.	You copy each bullet, watch a demo GIF the AI links, and you're done in 15 minutes.